

Approach to Implementation

The initial approach to the creation of the game stemmed from the control flow and functionalities of LibGDX for Java. To accomplish our goals for the game, we required three main sections:

1: A main game class that is initialized at runtime. In our implementation, this class is not responsible for logic, but for the initialization of resources needed for operation, and for calling to the rendering/resizing of screens behind the scenes.

2: The screens that are actually rendered. Screens implement the Screen interface provided by LibGDX, which are rendered by the game class if they are set as the active screen. In our implementation, we created one screen for the main menu, and one for the game logic. These two screens also have access to the screen switching logic such that they may swap places.

3: The gameworld to be run by the game screen, and all of the classes below or adjacent to it. These classes are responsible for the actual logic of the game, either running main logic in the game or being an actor within the logic.

In terms of our halfway point, we defined it as having a working map, coupled with a movable character and working collisions. Furthermore, each class should have its skeletons ready. Out of these, we were able to implement everything aside from working collisions. While we weren't able to meet our goals in time, it gave us the motivation to become more productive, which was reflected in both our meetings and our work ethic.

Modifications:

The original UML we created gave an overview of the main structure to the game rather than specific parts of the actual logic of implementation, so there were few changes to the original UML of the game outside of branching off from it. The first major change we needed was to extend a new abstract nonplayer class from the entity abstract class for all entities other than the player. This functionality was needed because certain entities needed logic for colliding with the player. The player was also moved into the game world from the screen. This change reflects the fact that we had overhauled the world system into a world selection model instead of continuous gameplay, such that each new world would need its own player instantiated. The transform was also overhauled into using a Vec2 class with x and y floats for its scale, position, and velocity. This made it much easier to work with, and to design functions for, as they could then operate on and/or return a proper coordinate/vector. The screen manager was also overhauled into a singleton, which allowed for the screens to directly pull the main instance and call to change screens without needing to call through the game class.

Roles:

Lane:

My initial role in this project was with the rough design of many high and low level components to the game through the UML, the control flow dictating how they should connect, and the rough implementation of the lower section of the UML, as well as the implementation of a wide variety of new automation processes to allow for easier implementations of other classes. Later into implementation, I mainly managed entity sprites/animations, game logic, merge conflicts, the floor drawing system, and other smaller subtasks

Rishon:

My role was to create the map, as well as implement the necessary game logic to ensure entities and objects were compatible within this map. To design the map, I found a 2D level editor application known as Tiled to be useful. Using this app was challenging as I was unfamiliar with the tile system, which ultimately required a lengthy process to understand. On the other hand, designing the map was also difficult as it required a lot of hands-on work; sprites for our game theme are not widely accessible. I used an application called Piskel to design sprites from scratch, and I used a website called PixelArtify to convert images that suited our game into a pixel art-style. Within the map, I was able to render the player, whose graphic was initially a red box for testing purposes, a common theme I followed before sprites were ready. I also added a functional camera which allows for smooth camera movement as the player moves. I implemented collisions, and made sure that the player and objects would glide smoothly into each other. This required a lot of tweaking in not only Tiled to ensure these collidable objects were placed exactly where they should be, but also in our game collision logic, where I ultimately decided to use a phantom box system. To explain, a fake box does a move first; if it has a collision, it shortens the move distance so the player stops exactly at the edge without getting stuck. I implemented static punishment entities, and made sure that if the player collides with these punishable cells, they face the designated punishment. Finally, I implemented the end cell system, which is only available once the player has satisfied the win condition. During this process, my biggest challenge was finding an optimal way to link Tiled and our Java game. My teammates had suggested using a JSON file to store the location of cells, but once I figured out how to extract the contents of the map onto our game, I decided this method was much more straightforward. This occurred around the halfway point of Phase 2, but from then on, it was smooth sailing.

Luke:

I was responsible for a variety of miscellaneous implementations, primarily the screen management system, the sound effects and music, and the user interface design. I started by creating the ScreenManager class, which created a singleton object to manage screen states and import the player data when required. I then created the main menu screen to allow the user to change settings, view their level progress, and enter the game. Using the basic paint software, paint.net, I drew a pixel art background that matched the theme of our game with a graph paper background and various school supplies. I also drew the buttons that I used throughout the

menus, and the background to be used by the in-game menus. To manage all the different collections of buttons, I created a series of layouts that each consisted of a button array, along with mappings of each button ID to a code block to execute when the button is clicked. The in-game menus function similarly to the main menu, with a popup screen that disables the world updates within the game to 'freeze time'. To implement the game's sound effects, I created an AudioManager class that stores each effect, and has triggers to loop, play, and end each sound. I also connected the functionality to the menu, to allow the user to toggle different sounds on and off. I gathered the majority of our sound effects and music from YouTube, and used the program Audacity to edit and cut the sounds to be the appropriate length and speed to fit the game. Implementing the sound triggers was a challenge, as it required adding checks to many different classes to communicate states such as movement, damage, and speed to the audio manager.

Sergei:

I was responsible for the implementation of the Eraser enemy class, which was responsible for the behavior and interaction with other objects. This class was also used as the basis for implementing other enemies classes. This class implements the logic of attacking the enemy. One of my main tasks was to implement a system to create a route from enemies to the player. The A* search algorithm was implemented, which relied on the Chebyshev distance heuristic, which finds the optimal path using only 8 directions of movement. The collision system has also been improved, which not only detects that a collision has occurred, but also takes action to resolve problems. It includes preventing multiple enemies from merging and passing through the player's texture.

Management:

In order to keep a tight control on management, we maintained a strict control over the creation and modifications to modules through a shared document, where we actively shared which modules we were currently working on, and a global changelog to the game. We used these systems in order to avoid miscommunication, and to avoid merge conflicts. Furthermore, to stay organized, we decided to have an online meeting through discord at least once a week. Here, we would discuss each other's progress, any potential problems we are facing, and reorganize to figure out what parts need to be completed next after each person has completed their responsibilities. Unfortunately, it was difficult to follow a routine for these meetings due to the fact our group had conflicting schedules. Therefore, we decided to meet whenever we all had an opening, which was usually on a Saturday or a Sunday. While minor concerns could be addressed through our Discord group chat, these meetings were reserved for bigger problems. Specifically, we followed a framework similar to Scrum, where we had week long sprints, and performed the planning phase for our next sprint during our meetings. This allowed us to maximize our productivity. On top of that, we would communicate each of the problems and concerns we faced. For instance, there were times when someone's implementation of a specific feature wasn't in line with another group member's expectations. To come to a shared

conclusion, the members would discuss the functionalities of their code, and from there it would be decided whether this feature should be changed or removed entirely. These types of situations would ensure we were all on the same page before advancing forward. Additionally, work was usually done in individual branches, so at the end of these meetings, a member would merge all branches and handle any merge conflicts while we were gathered. We made it a key point of interest that our game was running exactly as expected before we disbanded. Ultimately, following this methodology for our meetings allowed us to work in a smooth and coordinated way, and was the root cause in preventing many potential problems later down the line from arising.

External Libraries:

As discussed throughout the report, we are using the LibGDX library, which provides high level utilities for the control and rendering of the game. We decided to use this library in order to streamline the development of the game, as it provided us many useful utilities, primarily those in the rendering of the game.

Enhancing Code Quality:

During our development process, we would sometimes decide to cut corners and push something that just works. While we tried to keep the quality of code in mind, sometimes time crunches or frustrations caused our code to suffer. A direct result of this is it made our code much more difficult to read, so teammates couldn't go through each others' classes and easily gauge what was going on. At times it would also cause lag, creating a choppy experience for the user. To address these issues, we decided that once our game was fully complete, we'd go back and refactor to make our code much more logical.

One such example was to modularize our code, moving away from hardcoding behaviors into the main game loop and instead utilize inheritance structures. We see this directly in our Entity and Nonplayer inheritance structures, who ensured other classes such as our hazardous entities followed the same lifecycle. In the end, this allowed for our GameWorld class (which manages the world, its entities, and control over the game) to be much easier to manage.

While implementing the Menu screens, we quickly realized that stringing together if statements and declaring a new button at a specific x and y value for each layout would become incredibly unmaintainable and difficult to read. To solve this, we implemented an ENUM to store each menu layout, and each of those layouts corresponds to a subset of menu buttons. To streamline the process of creating new buttons, we added a create button function, that stores the y position of the previously created one, and adds a new one below it, with a constant amount of spacing. This significantly improved code quality, since it allowed for readable switch cases between layouts, and made buttons easy to rearrange and add.

Adding comments throughout this project was crucial for both communication between project members, as well as readability for future refactoring. The majority of comments were using Javadoc, which allowed us to create very detailed and exportable comments. Many of our classes have Javadoc headers that record the author and most recent update, and most functions provide a description and outline the parameters and return value in easy to understand terms. Javadoc is one of the biggest differences when it comes to code quality, as our detailed comments significantly improve the readability and maintainability of our entire codebase.

A final example of refactoring is the improvement of the classes of enemies Eraser and PencilSharpener. Initially, as development progressed, the Eraser class was created and the PencilSharpener class was based on it. They had a similar logic of finding a way to the player and moving. This led to duplication of most of the code of these classes and made debugging more difficult, so it was decided to create a new MobileEnemy class, from which the Eraser and PencilSharpener classes are inherited. Now they use the same and more reliable player search system, but they have different behavior when interacting with the player.

Biggest Challenges:

Members of our group were shaped by differing coding backgrounds, so each of us faced unique challenges. This was especially prevalent in the early stages, however members with prior experience were able to steer the direction of our project and keep everyone on the same page. Aside from that, this was our first time working together in a group setting. Hence, communication and figuring out group roles was a big barrier. Despite this, we had some group members take initiative and as a result the roles were naturally formed. Coordination also proved to be a hurdle, as it was difficult to book consistent meetings with members who had conflicting schedules. This fragmentation often led to long periods of silence where team members would work in isolation, without communicating their ideas and updates until the actual meeting. It also led to some group members unsure of their responsibilities, causing them to delay until the meeting occurs and only begin working once they understand the duties that have been assigned to them. Because we were so unsynchronized, it directly impacted our productivity during the first half of the term; we frequently found ourselves leaving substantial portions of the workload until the last second. A clear consequence of this was our inability to meet the goals we had set up for the March 5th halfway point. This was a necessary wake-up call for the group, because after this point everyone realized our current strategy was not working. We recognized that our early meetings were largely unstructured and unproductive, often ending without a clear path forward. Responsibilities were not clearly given, so many group members were left confused as to what they should be doing. Hence, we pivoted to a more purposeful meeting style, where we used our limited time to discuss what we've done, who does what concrete actionables next, and discussing all potential problems in our code while we had the chance, with a focus on merge conflicts. This change allowed for massive boosts in our productivity.